



Log In ▾

The wiki page is under active construction, expect bugs.

Programování v jazyce JAVA: vlastnosti a koncepce jazyka. Principy objektového programování.

vector2 ▾

B0B36PJV [Webové stránky předmětu](#)

- **Vývojové prostředí** – JDK, JVM, kompilace a běh programu, správa paměti, GC, profilování a optimalizace.
- **Objekty, třídy a jejich vztahy** – princip abstrakce a zapouzdření, modifikátory přístupu. Interface a abstraktní třída. Dědičnost a kompozice, polymorfismus, dynamická vazba.
- **Výčtové typy** – práce s kolekcemi, vzor iterátor, generické typy.
- **Vnitřní a anonymní třídy** – imutabilita, vzor singleton. Proměnné a metody třídy vs. instance.
- **Mechanismus výjimek** – typy a jejich ošetření, vlastní výjimky. Práce se soubory – přístup k souboru, textové vs. binární, proudy, ukládání dat. Sokety – typy soketů, typy spojení, síťová komunikace.
- **Paralelismus** – vícevláknové aplikace, problém souběhu a zastavení. Tvorba vláken a jejich ukončení, threadpool, synchronizace, volatilita.

1. Vývojové prostředí – JDK, JVM, kompilace a běh programu, správa paměti, GC, profilování a optimalizace

JDK, JRE, JVM

Většinu programovacích jazyků je možné rozdělit do dvou kategorií:

- **kompilované** – program je zkompilovaný do spustitelného souboru, skládá se z nativních procesorových instrukcí, které přímo vykonává procesor a musí být tedy kompilován pro konkrétní architekturu a systém. Mezi tyto jazyky patří například C/C++.
- **interpretované** – program není zkompilovaný, pro spuštění je nutný jiný program (interpreter), který program při spuštění převádí do nativních procesorových instrukcí, které může počítač vykonávat. Typický interpretovaný jazyk je třeba Python.

Java je možné zařadit do obou kategorií – program se první zkompile do **bytecode**, který je obecný – není specifický pro danou platformu. Pro tento proces je potřeba **JDK** (java development kit).

Tento **bytecode** je možné následně pustit s **JRE** (java runtime environment- subset **JDK**), který **bytecode** za běhu interpretuje.

Proces kompilace a spuštění v Javě:

- Zdrojový kód se píše do souborů s příponou `.java``.
- Překladač ``javac`` přeloží `.java`` do `.class`` souboru s bytecode.
- Bytecode spouští JVM pomocí příkazu ``java NázevTřídy``.

Java tedy umožňuje program zkompilevat pouze jednou do **bytecode** a následně spustit na jakékoliv platformě s **JRE** (i když v praxi to obecně tak jednoduché není – compile once, debug everywhere). Tento přístup vede k rychlosti blížíci se kompilovaným jazykům bez nutnosti kompilovat pro každou platformu zvlášť.

JVM (java virtual machine) je obsažen v **JRE** i **JDK** a je zodpovědný za překlad **bytecode** do nativních procesorových instrukcí.

JAR

JAR (Java ARchive) je archivní formát pro sdružení více tříd (*.class*) a dalších souborů do jednoho balíčku. Používá se k distribuci a spouštění Java aplikací.

Součástí může být také soubor `MANIFEST.MF`, ve kterém lze určit hlavní třídu programu:

- např. `Main-Class: com.example.Main`
- Umožňuje poté spustit aplikaci jednoduše: `java -jar program.jar`

Správa paměti

Stack

Malý paměťový prostor, ve kterém se typicky ukládají lokální proměnné (což obsahuje i reference, atd). Funguje na principu **last-in first-out**. Při zavolání funkce se na stacku alokuje místo pro všechny lokální proměnné dané funkce. Implementováno přes **stack-pointer**, který ukládá adresu konce stacku.

Pro každé volání metody vzniká tzv. **aktivní záznam**, který obsahuje:

- návratovou adresu,
- parametry metody,
- lokální proměnné.

Po ukončení metody se tento záznam odstraní.

Heap

Velký paměťový prostor, dynamicky alokovaná paměť. Typicky se zde ukládají objekty a větší datové struktury. Alokována pomocí **new**. O dealokaci se nestará programátor, ale **Garbage Collector (GC)**.

Heap je sdílený mezi všemi vlákny programu. Objekty zde zůstávají tak dlouho, dokud na ně existuje reference – poté je odstraní GC.

GC

Garbage Collector běží na pozadí **JVM**. Běží ve vlastním vlákně (nebo i více vláknech), takže neblokuje běh samotného programu (asynchronní). Drží si seznam všech referencí na data na **heap** a jakmile zjistí, že už žádné reference neexistují, tak data uvolní (dealokuje).

Garbage Collector sice nebude nikdy tak efektivní jako korektní manuální dealokace jako např. v C/C++, ale zjednodušuje psaní programu, jeho udržitelnost a značně snižuje riziko memory leaků kvůli špatným manuálním dealokacím.

Profilování a optimalizace

JDK a z části i **JRE** implementuje několik nástrojů pro profilování programu, jako třeba vytížení procesoru a jednotlivých vláken, využití paměti, stav GC.

JDK provádí mnoho optimalizací již při kompilaci **bytecode**, jako např. eliminace nepoužitého kódu, inlining, atd... Následně probíhá i několik optimalizací při interpretaci programu, jako např. cachování kompilace do nativních procesorových instrukcí.

Java nabízí několik nástrojů pro analýzu výkonu:

- `jconsole` – vizualizace běhu JVM (paměť, CPU, vlákna, GC).
- `jvisualvm` – pokročilé grafické rozhraní s možností sledovat heap, GC, threaddump, apod.
- `Java Flight Recorder` – nástroj pro detailní záznam a analýzu výkonu aplikace.
- `javap` – nástroj pro prohlížení bytecode (.class).

Tyto nástroje umožňují najít místa s největším využitím CPU, paměti nebo nejdelším časem běhu.

2. Objekty, třídy a jejich vztahy

Objektově orientované programování (OOP) je paradigma, ve kterém modelujeme svět pomocí objektů – entit, které kombinují data a chování. Základními principy OOP jsou: **abstrakce**, **zapouzdření**, **dědičnost** a **polymorfismus**.

Třída – abstraktní popis objektu, definuje jeho strukturu (atributy) a chování (metody). **Objekt** – konkrétní instance třídy, která nese data a reaguje na operace definované třídou.

Třída je šablona pro vytváření tříd, objekt je konkrétní instance třídy.

Například:

```
// class definition
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }
}

// object instantiation
Person person = new Person("John", 30);
```

Abstrakce a zapouzdření

Abstrakce – organizujeme koncepty do tříd, skrýváme vnitřní implementaci a vystavujeme jen potřebné rozhraní.

Zapouzdření – každý objekt chrání svůj stav a komunikuje s okolím přes veřejné metody.

Modifikátory přístupu v Javě:

- `private` – přístup pouze uvnitř třídy.
- `protected` – přístup z třídy, dědiců a stejného balíčku.
- `public` – přístup odkudkoliv.
- výchozí (bez modifikátoru) – přístup v rámci balíčku.

Dědičnost a kompozice

Dědičnost – třídám lze předat chování a atributy z nadřazené třídy (`extends`), vytváří hierarchii typů („is-a“ vztah).

Kompozice – třída obsahuje jiné objekty jako své atributy („has-a“ vztah), vhodná pro sdružování funkcionality.

Příklad rozdílu:

```
// dědičnost
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal { void speak() { System.out.println("Woof"); } }

// kompozice
class Engine { void start() { ... } }
class Car {
    private Engine engine = new Engine();
    void start() { engine.start(); }
}
```

Polymorfismus a dynamická vazba

Polymorfismus – schopnost objektu zareagovat různě podle svého konkrétního typu, i když je používán přes obecný typ.

Např. metoda `draw()` může mít jinou implementaci ve třídách `Circle`, `Square`, ale lze je volat skrze rozhraní `Shape`.

Dynamická vazba (late binding) – konkrétní metoda se určuje až v době běhu na základě typu objektu.

Rozdíl:

- *Přetížení (overloading)* – metoda se liší počtem nebo typem parametrů (rozhoduje se v čase překladač).
- *Přepisování (overriding)* – metoda se předefinuje v podtřídě (rozhoduje se v době běhu).

Interface a abstraktní třída

Rozhraní (interface) definuje sadu metod, které třída musí implementovat (``implements``). Třída může implementovat více rozhraní.

Abstraktní třída (``abstract``) může obsahovat jak implementované, tak neimplementované (abstraktní) metody. Třída ji může pouze dědit (``extends``).

Rozdíly:

- Rozhraní – bez stavových atributů, vhodné pro vícenásobnou dědičnost.
- Abstraktní třída – umožňuje sdílení částečné implementace a stavu.

Bonus: Vícenásobná dědičnost

Java neumožňuje vícenásobnou dědičnost tříd, aby se předešlo problémům jako je **diamond problem** – tedy situace, kdy by podtřída dědila stejnou metodu z více nadtříd a nebylo by jasné, kterou implementaci použít.

Místo toho umožňuje vícenásobnou dědičnost **rozhraní**:

```
interface A { void doA(); }
interface B { void doB(); }

class C implements A, B {
    public void doA() { System.out.println("A"); }
    public void doB() { System.out.println("B"); }
}
```

Bonus: Single a double dispatch

Single dispatch znamená, že metoda, která se má zavolat, se vybírá podle *dynamického typu objektu*, na kterém je metoda volána:

```
class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal { void speak() { System.out.println("Woof"); } }

Animal a = new Dog();
a.speak(); // zavolá se Dog.speak() – podle skutečného typu objektu
```

Double dispatch znamená, že metoda je vybrána na základě *dynamických typů dvou objektů*. V Javě se běžně simuluje pomocí návrhového vzoru **Visitor**:

```
interface Visitor {
    void visit(Dog d);
    void visit(Cat c);
}

interface Animal {
    void accept(Visitor v);
}

class Dog implements Animal {
    public void accept(Visitor v) { v.visit(this); }
}

class Cat implements Animal {
    public void accept(Visitor v) { v.visit(this); }
}
```

Zde metoda ``visit(...)`` závisí jak na typu návštěvníka (``Visitor``), tak na konkrétním typu zvířete (``Dog``, ``Cat``).

3. Výčtové typy, kolekce, iterátor, generické typy

Výčtové typy (enum)

Výčtový typ (angl. enumeration) je datový typ, který může nabývat pouze předem daných hodnot. Zvyšuje čitelnost a typovou bezpečnost programu – místo čísel nebo řetězců použijeme symbolická jména.

Například:

```
public enum Day {  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY,  
    SUNDAY  
}  
  
Day today = Day.MONDAY;
```

Výčtové typy jsou ve skutečnosti speciální typ třídy – lze do nich přidat:

1. konstruktor,
2. metody,
3. pole.

```
public enum Suit {  
    CLUBS(Color.BLACK),  
    DIAMONDS(Color.RED),  
    HEARTS(Color.BLACK),  
    SPADES(Color.RED);  
  
    private Color color;  
  
    Suit(Color c) {  
        this.color = c;  
    }  
  
    public Color getColor() {  
        return color;  
    }  
  
    public boolean isRed() {  
        return color == Color.RED;  
    }  
}
```

Kolekce (Java Collection Framework)

Java Collections Framework (JCF) je sada rozhraní a tříd pro práci s obecnými datovými strukturami.

Základní vlastnosti:

- umožňují efektivní ukládání, vyhledávání a zpracování objektů,
- hierarchicky uspořádané – Collection, List, Set, Map, Queue atd.

Základní typy kolekcí:

- List – uspořádaná kolekce s indexy (ArrayList, LinkedList)
- Set – množina bez duplicit (HashSet, TreeSet)
- Map – páry klíč–hodnota (HashMap, TreeMap)
- Queue – fronta (LinkedList, PriorityQueue)

Příklad:

```
List<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");
```

Iterátor

Iterátor je objekt, který umožňuje bezpečně procházet kolekce bez znalosti jejich vnitřní struktury.

Použití:

```
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Zkrácený zápis pomocí for-each:

```
for (String name : names) {
    System.out.println(name);
}
```

Rozhraní Iterable umožňuje použít kolekci ve for-each smyčce.

Poznámka: Kolekce nelze bezpečně měnit během iterace jinak než metodou `it.remove()`. Jinak hrozí `ConcurrentModificationException`.

Generické typy

Generické typy (generika) umožňují psát obecný, opakovatelný a typově bezpečný kód, který lze aplikovat na různé datové typy bez nutnosti přetypování.

Deklarace:

```
List<Integer> numbers = new ArrayList<>();
numbers.add(10);
```

Výhody generik:

1. typová bezpečnost,
2. žádné přetypování,
3. opakovatelnost kódu.

Generická metoda:

```
public static <T> void printAll(List<T> list) {
    for (T item : list) {
        System.out.println(item);
    }
}
```

Poznámka: Generika jsou v Javě implementována pomocí type erasure – typová informace se za běhu ztrácí (např. nelze napsat `new T()`).

4. Vnitřní a anonymní třídy, Imutabilita, Vzor singleton, Proměnné a metody třídy vs. instance

Vnitřní třídy

Vnitřní třída (inner class) je třída, která je definovaná uvnitř jiné třídy. Má přístup ke všem členům (včetně privátních) vnější třídy a typicky slouží jako pomocná třída.

Je-li vnitřní třída označena jako `static`, jedná se o statickou vnitřní třídu, která nemá přístup k instanci vnější třídy.

Výhodou je lepší zapouzdření a přehlednost kódu – vnitřní třídy se používají tam, kde nemá smysl jejich existence samostatně.

```
class OuterClass {
    int x = 10;

    class InnerClass {
        int y = 5;
    }
}
```

Anonymní třídy

Anonymní vnitřní třída je bezejmenná třída, která je definována a zároveň instanciována na místě. Používá se tam, kde potřebujeme jednorázově implementovat nějaké rozhraní nebo rozšířit třídu.

Typicky se využívá např. pro callbacky nebo posluchače událostí.

Anonymní třída je třída vnitřní bez názvu, pro kterou je vytvořen pouze jediný objekt.

GeeksforGeeks 

```
// Interface
interface Age {
    int x = 21;
    void getAge();
}

class AnonymousDemo {

    // Main driver method
    public static void main(String[] args)
    {

        // A hidden inner class of Age interface is created
        // whose name is not written but an object to it
        // is created.
        Age obj1 = new Age() {

            @Override public void getAge()
            {
                // printing age
                System.out.print("Age is " + x);
            }
        };

        obj1.getAge();
    }
}
```

Imutabilita

Imutabilní objekt je takový objekt, jehož stav po vytvoření již nelze změnit. To znamená:

- všechny atributy jsou private a final,
- objekt neposkytuje žádné metody, které by umožnily změnu stavu,
- objekt nevrací reference na své vnitřní proměnné.

Výhody:

- bezpečnost při vícevláknovém přístupu,
- snazší ladění a testování.

Vzor Singleton

Singleton je návrhový vzor, který zajišťuje, že daná třída má právě jednu instanci. Toho se docílí:

- soukromým (private) konstruktorem,
- statickou proměnnou pro instanci třídy,
- veřejnou statickou metodou pro přístup k instanci.

```
public class Singleton {
    private static final Singleton instance = new Singleton();

    private Singleton() {}

    public static Singleton getInstance() {
        return instance;
    }
}
```

Proměnné a metody třídy vs. instance

- Instanční proměnné – náleží konkrétní instanci třídy, každá instance může mít jiné hodnoty.
- Třídní proměnné (statické) – sdílené všemi instancemi, existují nezávisle na instanci.
- Instanční metody – pracují s instančními daty, volají se na konkrétních objektech.
- Statické metody – patří třídě, volají se bez instance, nemají přístup k nestatickým členům.

5. Mechanismus výjimek

Výjimka (*exception*) je **neočekávaná událost** během běhu programu. Znemožňuje dokončení běžného toku programu, pokud není správně ošetřena. Výjimky se typicky používají pro chybové stavy, jako je dělení nulou nebo přístup mimo rozsah pole.

Hierarchie výjimek v Javě

* Všechny výjimky jsou potomky třídy `Throwable`.

- `Error` – závažné chyby JVM (např. `OutOfMemoryError`). Nejsou určeny k zachytávání.
- `Exception` – výjimky, které lze (a měly by se) ošetřit.
 - `RuntimeException` – běhové výjimky (např. `NullPointerException`, `IllegalArgumentException`).

Checked vs. Unchecked výjimky

- **Checked** – kompilátor hlídá; musíš je buď zachytit pomocí `catch`, nebo předat dál klíčovým slovem `throws` (např. `FileNotFoundException`).
- **Unchecked** – všechny výjimky odvozené od `RuntimeException` a také `Error`; kompilátor jejich ošetření nevynucuje (např. `NullPointerException`).

Ošetření výjimek

Výjimky se ošetřují pomocí konstrukcí `try-catch-finally`.

```
try {  
    // rizikový kód  
} catch (IOException | SQLException ex) { // multi-catch od Java 7  
    e.printStackTrace(); // ošetření výjimky  
    log(ex);  
    throw ex;           // volitelné předání dál  
} finally {  
    cleanup();          // provede se vždy  
}
```

- `catch` může být více, podle typu výjimky.
- `finally` se vždy vykoná – i když dojde k výjimce nebo `return`.

Try-with-resources

Od Javy 7 existuje `try-with-resources`, který automaticky zavře prostředky (např. soubory, streamy) implementující `AutoCloseable`.

```
try (BufferedReader br = Files.newBufferedReader(Path.of("data.txt"))) {  
    return br.readLine();  
}
```

Vlastní výjimky

Vlastní výjimky se vytvářejí děděním ze třídy `Exception` nebo `RuntimeException`.

```
public class MyException extends Exception {  
    public MyException() {}  
    public MyException(String msg) { super(msg); }  
    public MyException(String msg, Throwable cause) { super(msg, cause); }  
}
```

- **Kdy se hodí?** Když stávající výjimky nedostačují a potřebuješ popsat specifickou situaci.
- Rozhodni se, zda má být **checked** (nutí volající k ošetření) nebo **unchecked** (dědí z `RuntimeException`).
- Pokud dědíme z `Exception`, jedná se o checked výjimku – musí být ošetřena.
- Pokud dědíme z `RuntimeException`, jedná se o unchecked výjimku – nemusí být ošetřena.

Best practices

- Zachytávej co nejkonkrétnější typy výjimek, až poté obecné (`Exception`, `Throwable`).
- Nepoužívej prázdné `catch` bloky – vždy loguj nebo informuj uživatele.
- Uvolňuj prostředky pomocí `finally` nebo `try-with-resources`.
- Přidávej popisné chybové zprávy (`new IOException("Soubor nenalezen")`).

- Vlastní výjimky používej pro specifické, smysluplné chyby v aplikaci.

Shrnutí podle typu výjimky

* `Throwable` – kořenová třída, zachytitelná, ale neměla by se běžně používat. * `Exception` – standardní ošetřitelné výjimky.

- `IOException`, `SQLException`, `ParseException` – checked výjimky.
- `RuntimeException`, `NullPointerException`, `IndexOutOfBoundsException` – unchecked.

* `Error` – závažné chyby – většinou se nezachytávají (`OutOfMemoryError`, `StackOverflowError`).

Práce se soubory (java.io)

Soubor je množina údajů uložená ve vnější paměti. Přístup k němu probíhá pomocí proudů (streamů), které umožňují čtení a zápis dat sekvenčně nebo náhodně. Dělíme je podle typu dat na textové a binární.

Klíčové třídy

- **File** – reprezentuje cestu k souboru nebo adresáři. Lze ověřit existenci, práva, velikost, vytvořit složky/soubory.
- **InputStream / OutputStream** – bajtové proudy (pro binární data).
 - Dekorátory: `Buffered*` (vyrovnávací paměť), `Data*` (primitiva), `Object*` (serializace), `GZIP*` (komprese)...
- **Reader / Writer** – znakové proudy (pro textová data).
 - Často používané: `BufferedReader`, `InputStreamReader`, `FileWriter`.
- **RandomAccessFile** – umožňuje číst/zapisovat na libovolnou pozici (náhodný přístup).
- Rozhraní: `Closeable`, `Flushable`, `Serializable` – sjednocují chování při zavírání, flushování a serializaci objektů.

Otevření binárních proudů

```
try (InputStream in = new FileInputStream("logo.png");
    OutputStream out = new FileOutputStream("copy.png")) {

    byte[] buf = new byte[8192];
    int n;
    while ((n = in.read(buf)) != -1) {
        out.write(buf, 0, n);
    }
} // oba proudy se zavřou automaticky (try-with-resources)
```

Otevření a čtení textového souboru (java.io)

Nejčastější (a dodnes plně dostačující) kombinací je dvojice `FileReader` + `BufferedReader`:

```
try (BufferedReader br =
    new BufferedReader(new FileReader("soubor.txt"))) {

    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} // try-with-resources zajistí zavření souboru
```

- `FileReader` čte **znaky** (automaticky použije defaultní kódování platformy, nebo explicitně zadej např. `StandardCharsets.UTF_8`).
- `BufferedReader` obalí čtení do větších bloků → méně systémových volání, vyšší výkon.
- Konstrukce **try-with-resources** (Java 7+) zavře reader i v případě výjimky.

Textové vs. binární soubory

- **Textové** – čitelné pro člověka, pracujeme se znaky pomocí Reader/Writer. Nutné správné kódování (UTF-8, ASCII...).
- **Binární** – rychlejší, nečitelné, používáme InputStream/OutputStream. Vhodné pro obrázky, audio, objekty apod.

Serializace objektů

```
try (ObjectOutputStream oos =
    new ObjectOutputStream(new FileOutputStream("state.bin"))) {
    oos.writeObject(gameState);
}
```

- Třída musí implementovat `Serializable`.
- Lze uložit i více objektů. Pozor na kompatibilitu verzí tříd (pole `serialVersionUID`).

Přístup k datům

- **Sekvenční** – čtení/zápis po sobě jdoucích bajtů/řádků (většina proudů).
- **Náhodný (Random Access)** – přímý přístup k určitému místu v souboru (`RandomAccessFile`).

Cesty k souborům

- **Absolutní** – začíná kořenem systému (např. `/home/user/file.txt` nebo `C:\Users\file.txt`).
- **Relativní** – relativní k pracovnímu adresáři aplikace.

Best practices

- Vždy používej `try-with-resources` – eliminuje riziko nezavření souboru.
- Při práci s textem dej pozor na kódování (UTF-8 doporučeno).
- Buďruj vstupy/výstupy pomocí `Buffered*` – zvyšuje výkon.
 - Nezachytávej holou `Exception`; loguj a přeposílej konkrétní typy.
 - Při sériové práci se soubory preferuj nové NIO API (`java.nio.file.Path`) – ošetří limity `File` a umí asynchronní I/O, ale porozumění `java.io` je nutný základ.

Sokety

Soket je objekt, který propojuje aplikaci se síťovým protokolem. Umožňuje síťovou komunikaci mezi dvěma koncovými body (např. klientem a serverem) prostřednictvím definovaných API. V Javě se sokety používají pro komunikaci přes protokoly TCP a UDP.

Typy soketů a API

Třída	Protokol	Povaha	Typická použití
_____	_____	_____	_____
Socket / ServerSocket	TCP	spojované, spolehlivé	chat, přenos souborů
DatagramSocket	UDP	nespojované, best-effort	streaming, multiplayer hry
MulticastSocket	UDP multicast	skupinový přenos	discovery, video broadcast

TCP vs. UDP – typy spojení

TCP (Transmission Control Protocol)

- Spojovaný protokol – před komunikací naváže spojení
- Zaručuje doručení, pořadí zpráv a detekci chyb
- Pomalejší, ale spolehlivý

UDP (User Datagram Protocol)

- Nespojovaný protokol – žádné spojení, zprávy mohou být ztraceny nebo doručeny jinak
- Rychlý, nízká latence, žádné záruky
- Nutno si případně ošetřit potvrzení a opakování ručně

Ukázkový TCP server/klient

```
// === Server ===
try (ServerSocket srv = new ServerSocket(9000)) {
    while (true) {
        try (Socket s = srv.accept();
            BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
```

```

        PrintWriter out = new PrintWriter(s.getOutputStream(), true) {

            String line = in.readLine();
            out.println("Echo: " + line);

        }
    }
}

// === Klient ===
try (Socket s = new Socket("localhost", 9000);
    BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
    PrintWriter out = new PrintWriter(s.getOutputStream(), true)) {

    out.println("Ahoj světe");
    System.out.println(in.readLine());
}

```

Primitiva soketu

- **create** – vytvoření nového soketu
- **bind** – přiřazení lokální adresy a portu
- **connect** – navázání spojení (klient)
- **listen** – čekání na spojení (server)
- **accept** – přijetí spojení
- **send / receive** – odeslání a příjem dat
- **shutdown / close** – uzavření spojení

Best practices

- **Porty a backlog:** při `new ServerSocket(port, backlog)` nastavíš frontu nepřijatých spojení.
- Nastav `setSoTimeout()` → ochrana proti zablokování čtení.
- Na mnoho paralelních spojení použij **java.nio.channels + selektory** (non-blocking I/O).
- Pro šifrovaný přenos sáhní po `SSLSocket` / `SSLServerSocket` nebo moderním klientovi `java.net.http`.
- Vždy uzavírej soket v `try-with-resources` nebo `finally`.

6. Paralelismus

Paralelismus umožňuje efektivnější využití systémových prostředků (více jader CPU, paralelní zpracování vstupů, vyšší výkon, reakce na události). V Javě se používá vícevláknové programování pomocí třídy `Thread`, rozhraní `Runnable`, případně thread poolu (`ExecutorService`).

Nutné dodržet pravidla synchronizace, aby se zabránilo deadlocku.

Vytváření a spouštění vláken

Vláknem lze vytvořit buď děděním od třídy `Thread`, nebo implementací rozhraní `Runnable`:

Objekty jsou odvozené od třídy `Thread`, tělo nezávislého výpočtu definujeme v metodě `run()`.

```

public class Worker extends Thread {
    private final int numberOfJobs;

    public Worker(int id, int jobs) {
        super("Worker " + id);
        myID = id;
        numberOfJobs = jobs;
        stop = false;
        System.out.println("Worker id: " + id + " has been created threadID: " + getId());
    }

    public void run() {
        doWork();
    }
}

Worker thread = new Worker(1, 10);
thread.start(); //new thread is created
System.out.println("Program continues here");

```

Pokud nelze použít dědění od `Thread` implementujeme rozhraní `Runnable`:

```

public class WorkerRunnable implements Runnable {
    private final int id;
    private final int numberOfJobs;

    public WorkerRunnable(int id, int jobs) {
        this.id = id;
        numberOfJobs = jobs;
    }

    public String getName() {
        return "WorkerRunnable " + id;
    }

    @Override
    public void run() { ... }
}

WorkerRunnable worker = new WorkerRunnable(1, 10);
Thread thread = new Thread(worker, worker.getName());
thread.start();

public void run() {
    Thread thread = Thread.currentThread();

    for (int i = 0; i < numberOfJobs; ++i) {
        System.out.println("Thread name: " + thread.getName());
    }
}

```

Synchronizace a problémy souběhu

Při práci s více vlákny vznikají typické problémy:

- Race condition – více vláken přistupuje ke sdíleným datům bez koordinace. Výsledek je nedefinovaný.
 - Řešení: synchronized bloky, volatile proměnné
- Deadlock – dvě nebo více vláken čekají na zdroje, které drží jiná vlákna.
 - Řešení: správné pořadí zamykání, timeouty, algoritmy prevence deadlocku
- Producer/Consumer – problém koordinace mezi vlákny produkujícími a spotřebovávajícími data (např. fronta).
 - Řešení: wait(), notify(), fronty z balíku java.util.concurrent
- Reader/Writer – vícero vláken čte a/nebo zapisuje do stejného objektu.
 - Řešení: ReadWriteLock nebo rozdělení na operace pouze pro čtení a zápis

Synchronized, Volatile, Join

- synchronized – zajišťuje, že kód (kritická sekce) provádí v daném čase jen jedno vlákno
- volatile – proměnná může být měněna z více vláken, JVM nezachová její hodnotu v cache (když se změní na false - informuje to ostatní vlákna, ale ne jako counter)
- join() – čeká na dokončení jiného vlákna
- wait() / notify() – vlákna čekají a signalizují si mezi sebou (pouze uvnitř synchronized bloku)

Thread Pool (Executor)

Používá se místo ručního vytváření velkého počtu vláken. Efektivnější správa a recyklace.

```

ExecutorService executor = Executors.newFixedThreadPool(4);
executor.submit(new WorkerRunnable(1, 10));
executor.shutdown();

```

- ExecutorService – spravuje vláknový pool
- submit() – spustí úkol
- shutdown() – ukončí Executor
- Future – objekt s výsledkem nebo stavem asynchronního výpočtu

Kdy použít paralelismus

- Více úloh může běžet souběžně – více jádrový procesor
- Aplikace provádí dlouhé IO operace, ale musí zůstat responzivní (např. GUI)
- Kontrola nebo úkoly na pozadí (např. notifikace, synchronizace)

